Fatec
Ipiranga
Pastor Eneas Tognini
.
TADS - Análise Desenvolvimento de Sistemas

Programacao Estruturada e Modular

*Ponteiros

Prof.: Veríssimo - carlos.pereira01@cps.sp.gov.br

17/04/2026

Sumário

1 Conceituando Ponteiros **2**

2 Declaração e Operadores Fundamentais **2**

2.1 O Operador de Referência (&) 2

2.2 O Operador de Desreferenciação (*) 2

3 Manipulações Básicas **3**

4 Ponteiros e Arrays: Uma Relação Íntima **4**

4.1 Aritmética de Ponteiros 4

4.2 Exemplo: Notação de Array vs. Notação de Ponteiro 4

4.3 Arrays como Parâmetros de Funções 4

4.4 Diferença Fundamental 5

5 A Verdade sobre Arrays em Funções **6**

5.1 O Fenômeno do Pointer Decay 6

5.2 Consequências Práticas 6

6 Passagem de Parâmetros: Valor vs. Referência **7**

6.1 Passagem por Valor (Sem Ponteiro) 7

6.2 Passagem por Referência (Com Ponteiro) 7

6.3 Por que usar Referência? 8

7 Resumo de Operadores **8**

1 Conceituando Ponteiros

Um **ponteiro** é uma variável especial que, em vez de armazenar um valor comum (como um inteiro ou um caractere), armazena um **endereço de memória**.

Imagine a memória do computador como uma grande rua com várias casas numeradas. Uma variável comum é o conteúdo dentro da casa. Um ponteiro é um papel que contém o número (endereço) dessa casa.

2 Declaração e Operadores Fundamentais

Para declarar um ponteiro, utilizamos o operador asterisco (*) antes do nome da variável.

2.1 O Operador de Referência (&)

O operador `&` (e comercial) é usado para obter o **endereço** de uma variável existente.

2.2 O Operador de Desreferenciação (*)

Quando usado na declaração, o * indica que a variável é um ponteiro. Quando usado no corpo do código (antes do nome de um ponteiro já existente), ele acessa o **valor** armazenado no endereço que o ponteiro carrega.

```c
#include <stdio.h>

int main() {
    int numero = 42;     // Variavel comum
    int *p;              // Declaracao de um ponteiro para inteiro

    p = &numero;         // p recebe o ENDERECO de numero

    printf("Valor de numero: %d\n", numero);
    printf("Endereco de numero: %p\n", &numero);
    printf("Valor armazenado em p: %p\n", p);
    printf("Valor apontado por p (*p): %d\n", *p);

    return 0;
}
```

Listing 1: Exemplo de Declaração e Acesso

3 Manipulações Básicas

É crucial distinguir quatro informações distintas ao trabalhar com ponteiros:

- **Valor da variável apontada:** O dado real (ex: 42).
- **Endereço da variável:** Onde ela mora na RAM.
- **Valor armazenado no ponteiro:** O endereço que ele "vigia".
- **Valor apontado pelo ponteiro:** O conteúdo do endereço que ele carrega.

```c
#include <stdio.h>

int main() {
    int x = 10;
    int *ptr = &x;

    printf("Antes: x = %d\n", x);

    // Alterando o valor de x atraves do ponteiro
    *ptr = 20;

    printf("Depois: x = %d\n", x);
    printf("Valor via ponteiro: %d\n", *ptr);

    return 0;
}
```

Listing 2: Manipulando valores através de ponteiros

4 Ponteiros e Arrays: Uma Relação Íntima

Em C, existe uma equivalência muito forte entre arrays e ponteiros. Quando declaramos um array como `int vet[5]`, o identificador `vet` comporta-se, na maioria das expressões, como um **ponteiro constante** que aponta para o endereço do primeiro elemento (`&vet[0]`).

4.1 Aritmética de Ponteiros

Ao somar 1 a um ponteiro, o compilador não soma apenas 1 byte, mas sim o tamanho do tipo de dado que o ponteiro aponta (ex: 4 bytes para um `int`). Isso nos permite navegar pelo array de forma eficiente.

- `vet` → Aponta para o início do array.
- `vet + 1` → Aponta para o segundo elemento.
- `*(vet + i)` → É exatamente o mesmo que escrever `vet[i]`.

4.2 Exemplo: Notação de Array vs. Notação de Ponteiro

O código abaixo demonstra que podemos percorrer e manipular um array sem nunca utilizar os colchetes `[]`.

```c
#include <stdio.h>

int main() {
    double notas[4] = {7.5, 8.0, 9.5, 6.0};
    double *p = notas; // 'notas' ja e um endereco

    printf("Acessando via Indices:\n");
    for(int i = 0; i < 4; i++) {
        printf("Nota %d: %.1f\n", i, notas[i]);
    }

    printf("\nAcessando via Ponteiros (Aritmetica):\n");
    for(int i = 0; i < 4; i++) {
        // *(p + i) desloca o ponteiro i posicoes e desreferencia
        printf("Endereco: %p | Valor: %.1f\n", (p + i), *(p + i));
    }

    return 0;
}
```

Listing 3: Percorrendo Arrays com Ponteiros

4.3 Arrays como Parâmetros de Funções

Sempre que você passa um array para uma função, você está, na verdade, passando um ponteiro. É por isso que alterações no array dentro de uma função são refletidas fora dela.

```c
#include <stdio.h>

// Ambas as declaracoes abaixo sao equivalentes para o compilador:
// void zerar(int vet[])  OU  void zerar(int *vet)
void zerar(int *ptr, int tamanho) {
    for(int i = 0; i < tamanho; i++) {
        *(ptr + i) = 0; // Zera o valor no endereco apontado
    }
}

int main() {
    int numeros[] = {10, 20, 30, 40, 50};
    int tam = 5;

    zerar(numeros, tam);

    printf("Primeiro elemento apos a funcao: %d\n", numeros[0]);
    return 0;
}
```

Listing 4: Processando Arrays em Funcoes

4.4 Diferença Fundamental

Embora funcionem de forma parecida, há uma diferença vital:

1. Um **Ponteiro** é uma variável: você pode fazer `p = x` ou `p++`.
2. Um **Array** é um identificador constante: você **não** pode fazer `notas = x` ou `notas++`. O endereço base de um array é fixo.

5 A Verdade sobre Arrays em Funções

Uma dúvida comum é: "Ao passar um array para uma função, o C cria uma cópia de todos os elementos?". A resposta é **não**. Por questões de eficiência de memória, a Linguagem C sempre passa apenas o **endereço do primeiro elemento**.

5.1 O Fenômeno do Pointer Decay

Quando o nome de um array é passado para uma função, ele "decai"para um ponteiro. Isso significa que a função não recebe um novo array, mas sim uma cópia do endereço de memória onde o array original começa.

```c
#include <stdio.h>

void analisarArray(int arr[]) {
    // sizeof(arr) retornara o tamanho do PONTEIRO (geralmente 8 bytes em 64-bit)
    // e nao o tamanho do array original (ex: 20 bytes para 5 ints).
    printf("Tamanho do parametro na funcao: %lu bytes\n", sizeof(arr));
}

int main() {
    int vet[5] = {1, 2, 3, 4, 5};

    printf("Tamanho do array original na main: %lu bytes\n", sizeof(vet));

    analisarArray(vet);

    return 0;
}
```

Listing 5: Provando que o Array e um Ponteiro na Funcao

5.2 Consequências Práticas

1. **Alteração Direta:** Como a função tem o endereço real (`&vet[0]`), qualquer alteração feita dentro da função via `arr[i]` modificará o array original na `main`.

2. **Perda da Informação de Tamanho:** A função não sabe quantos elementos o array possui, pois ela recebeu apenas um endereço. Por isso, **sempre** devemos passar o tamanho do array como um segundo parâmetro: `void processar(int *vet, int tam)`.

3. **Sintaxe Equivalente:** Para o compilador, as três assinaturas abaixo são idênticas:

   - `void func(int vet[])`
   - `void func(int vet[100])` // O 100 é ignorado pelo compilador!
   - `void func(int *vet)`

6 Passagem de Parâmetros: Valor vs. Referência

Para entender ponteiros, é fundamental compreender como a linguagem C gerencia a memória durante a chamada de funções.

6.1 Passagem por Valor (Sem Ponteiro)

Na passagem por valor, o programa cria uma **cópia** dos dados na pilha de execução (*stack*). Qualquer alteração feita dentro da função afeta apenas a cópia local, e não a variável original da função chamadora.

6.2 Passagem por Referência (Com Ponteiro)

Na passagem por referência, não enviamos o dado, mas sim o **endereço de memória** da variável original. Através da desreferenciação (*), a função consegue "enxergar"e modificar o conteúdo original diretamente no endereço onde ele reside.

```c
#include <stdio.h>

// Exemplo SEM PONTEIRO (Passagem por Valor)
// Uma copia e criada na memoria. O 'x' original nao muda.
void incrementarPorValor(int n) {
    n = n + 1;
    printf("Dentro da funcao (Valor): %d\n", n);
}

// Exemplo COM PONTEIRO (Passagem por Referencia)
// Recebemos o endereco. O '*' acessa o valor real.
void incrementarPorReferencia(int *p) {
    *p = *p + 1;
    printf("Dentro da funcao (Referencia): %d\n", *p);
}

int main() {
    int valor = 10;

    printf("--- Teste 1: Passagem por Valor ---\n");
    printf("Antes: %d\n", valor);
    incrementarPorValor(valor);
    printf("Depois: %d (Nao alterou!)\n\n", valor);

    printf("--- Teste 2: Passagem por Referencia ---\n");
    printf("Antes: %d\n", valor);
    // Passamos o endereco utilizando o operador '&'
    incrementarPorReferencia(&valor);
    printf("Depois: %d (Alterado com sucesso!)\n", valor);

    return 0;
}
```

Listing 6: Comparativo: Por Valor vs. Por Referência

6.3 Por que usar Referência?

1. **Modificação Direta:** Permite que funções retornem ou alterem múltiplos valores (visto que o `return` padrão só devolve um).
2. **Eficiência:** Evita a cópia de grandes estruturas de dados (como `structs` pesadas), economizando memória e processamento.
3. **Escopo:** É a única forma de uma sub-rotina manipular variáveis que pertencem ao escopo da `main` ou de outras funções.

7 Resumo de Operadores

Operador	Significado
`type *var`	Declara um ponteiro do tipo *type*.
`&var`	Retorna o endereço de memória de *var*.
`*ptr`	Acessa o conteúdo do endereço apontado por *ptr*.

Tabela 1: Referência rápida de ponteiros.